

ACARA 13. Controller

Pokok Bahasan	: Controller
Acara Praktikum	: MINGGU 4/1
Tempat	: Hybrid di Politeknik Negeri Jember
Alokasi Waktu	: 100 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami dan mengelola Controller di Laravel.
2. Mahasiswa mampu mengimplementasikan Controller dengan Routing.
3. Mahasiswa mampu menggunakan Middleware dalam Controller.
4. Mahasiswa mampu menggunakan Dependency Injection dalam Controller.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat Lunak Menganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami dan mengelola Controller di Laravel.
2. Mahasiswa mengimplementasikan Controller dengan Routing.
3. Mahasiswa menggunakan Middleware dalam Controller.
4. Mahasiswa menggunakan Dependency Injection dalam Controller.

d. Dasar Teori

Dalam Laravel, **Controller** adalah bagian dari **MVC (Model-View-Controller)** yang bertanggung jawab untuk menangani logika aplikasi dan interaksi antara Model (data) dan View (tampilan). Controller digunakan untuk mengelola permintaan HTTP dan mengembalikan respons kepada pengguna.

Dengan menggunakan Controller, kode menjadi lebih terstruktur, lebih mudah dipelihara, dan lebih reusable dibandingkan menulis semua logika dalam rute langsung (`routes/web.php` atau `routes/api.php`).

e. Alat dan Bahan

1. PC / Laptop
2. Windows 11
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Visual Studio Code
6. Kertas A4 / Folio Bergaris
7. Pulpen

f. Prosedur Kerja

1) Membuat Controller di Laravel

Untuk membuat Controller di Laravel, gunakan perintah Artisan berikut:

```
php artisan make:controller NamaController
```

Contoh:

```
php artisan make:controller ProductController
```

Perintah ini akan menghasilkan file controller di `app/Http/Controllers/ProductController.php`.

Jika Anda ingin membuat Controller dengan metode bawaan untuk operasi CRUD, gunakan flag `--resource`:

```
php artisan make:controller ProductController --resource
```

Ini akan membuat Controller dengan metode bawaan seperti `index()`, `store()`, `update()`, dll.

2) Struktur Dasar Controller

Setelah membuat Controller, berikut adalah contoh strukturnya:

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;

class BookController extends Controller{
    public function index(){
        return "Menampilkan semua produk";
    }

    public function show($id){
        return "Menampilkan produk dengan ID: " . $id;
    }

    public function create(){
        return "Form tambah produk";
    }
}
```

```

    }

    public function store(Request $request){
        return "Menyimpan produk baru";
    }

    public function edit($id){
        return "Form edit produk dengan ID: " . $id;
    }

    public function update(Request $request, $id){
        return "Mengupdate produk dengan ID: " . $id;
    }

    public function destroy($id){
        return "Menghapus produk dengan ID: " . $id;
    }
}

```

Menggunakan Controller dalam Routing

Untuk menghubungkan Controller dengan route, buka file routes/web.php atau routes/api.php, lalu tambahkan rute berikut:

a) Rute Biasa

```

use App\Http\Controllers\ProductController;
Route::get('/product', [ProductController::class, 'index']);
Route::get('/product/{id}', [ProductController::class, 'show']);
Route::post('/product', [ProductController::class, 'store']);
Route::put('/product/{id}', [ProductController::class, 'update']);
Route::delete('/product/{id}', [ProductController::class, 'destroy']);

```

b) Rute Resource (CRUD Otomatis)

Jika ingin mengatur semua rute CRUD sekaligus, gunakan:

```

Route::resource('product', ProductController::class);

```

Ini akan otomatis membuat rute berikut:

HTTP Method	URI	Method di Controller	Fungsi
GET	/book	index()	Menampilkan semua book
GET	/book/create	create()	Menampilkan form tambah book
POST	/book	store()	Menyimpan book baru
GET	/book/{id}	show()	Menampilkan book tertentu
GET	/book/{id}/edit	edit()	Menampilkan form edit book
PUT/PATCH	/book/{id}	update()	Mengupdate book tertentu
DELETE	/book/{id}	destroy()	Menghapus book tertentu

atau bisa dicek dengan perintah :

```
php artisan route:list
```

3) Middleware dalam Controller

Middleware digunakan untuk memberikan perlindungan atau validasi pada Controller. Anda bisa menerapkan Middleware dalam Controller seperti ini:

```
class BookController extends Controller
{
    public function __construct()
    {
        $this->middleware('auth');
        // Memastikan hanya user yang login yang bisa mengakses
    }

    public function index()
    {
        return "Menampilkan semua book";
    }
}
```

Atau menerapkannya hanya pada metode tertentu:

```
$this->middleware('auth')->only(['store', 'update', 'destroy']);
// Hanya untuk metode tertentu
```

```
$this->middleware('guest')->except(['index', 'show']);
// Kecuali metode tertentu
```

Controller dengan Dependency Injection

Dependency Injection digunakan untuk memasukkan class lain ke dalam Controller, misalnya Model Book:

```
use App\Models\Book;
use Illuminate\Http\Request;

class BookController extends Controller
{
    public function index(Book $book)
    {
        return $book->all(); // Mengambil semua book dari database
    }
}
```

API Controller (Tanpa View, Hanya JSON Response)

Jika ingin membuat Controller untuk API, gunakan perintah:

```
php artisan make:controller ApiProductController --api
```

Controller yang dibuat tidak memiliki create() dan edit(), karena API tidak membutuhkan tampilan.

Contoh implementasi API Controller:

```
use App\Models\Peoduct;
use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
class ApiProductController extends Controller
{
    public function index()
    {
        return response()->json(Product::all());
    }

    public function show($id)
    {
        return response()->json(Product::find($id));
    }
}
```

Rutenya:

```
Route::apiResource('product', ApiProductController::class);
```

g. Hasil dan Pembahasan

Format Laporan

https://docs.google.com/document/d/1FmnUT_6445Z7Ek8V1bqg0wfy8XVL_r3doVncwfuQfFU

- Kertas A4
- Format *.pdf
- Struktur Laporan: Disesuaikan dengan format yang diberikan dan ditambahkan foto resmi.
- Berikan Identitas Diri: NIM, Nama, Golongan sesuai yang tertera di Format
- Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
**Contoh: **ACARA-13_A_E1234356_BUDI.pdf
- Kumpulkan di Elearning : <https://slearn-jti.com/>

h. Kesimpulan

Dengan memahami dan mengelola Controller di Laravel, mengimplementasikan Controller dengan Routing, menggunakan Middleware dalam Controller, menggunakan Dependency Injection dalam Controller, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan langkah kerja kurang tepat dalam mengerjakan		V		
4	Keterlambatan waktu pengumpulan dan langkah kerja tidak sesuai	V			
Total Skor*		Skor x 25			

ACARA 14. Model

Pokok Bahasan	: Model
Acara Praktikum	: MINGGU 4/2
Tempat	: Hybrid di Politeknik Negeri Jember
Alokasi Waktu	: 100 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep model dalam laravel.
2. Mahasiswa mampu membuat dan mengelola model di laravel.
3. Mahasiswa mampu melakukan operasi database menggunakan model.
4. Mahasiswa mampu menerapkan relasi antar model dalam laravel.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat LunakMenganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami konsep model dalam laravel.
2. Mahasiswa membuat dan mengelola model di laravel.
3. Mahasiswa melakukan operasi database menggunakan model.
4. Mahasiswa menerapkan relasi antar model dalam laravel.

d. Dasar Teori

Dalam Laravel, **Model** adalah bagian dari pola **MVC (Model-View-Controller)** yang bertanggung jawab untuk berinteraksi dengan database. Model digunakan untuk merepresentasikan tabel dalam database dan menyediakan cara yang lebih mudah untuk mengakses, mengubah, dan mengelola data.

Laravel menggunakan **Eloquent ORM (Object-Relational Mapping)** untuk memudahkan interaksi dengan database. Dengan Eloquent, kita bisa bekerja dengan database menggunakan objek dan metode berbasis OOP (Object-Oriented Programming) daripada

harus menulis query SQL secara langsung.

e. Alat dan Bahan

1. PC / Laptop
2. Windows 11
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Visual Studio Code
6. Kertas A4 / Folio Bergaris
7. Pulpen

f. Prosedur Kerja

1) Cara Membuat Model di Laravel

Untuk membuat Model di Laravel, gunakan perintah Artisan berikut:

```
php artisan make:model NamaModel
```

Contoh:

```
php artisan make:model Product
```

Perintah ini akan membuat file `Product.php` di dalam direktori `app/Models/`.

Jika Anda ingin langsung membuat Model dengan Migration, Controller dan Resource, gunakan:

```
php artisan make:model Product -mcr
```

1. **-m (Migration)** akan otomatis membuat file migrasi pada direktori `database/migrations`.
2. **-c (Controller)** akan otomatis membuat file controller pada direktori `App\Http\Controllers`.
3. **-r (Resource)** akan otomatis mengisi file controller yang dihasilkan dengan metode CRUD (Create, Read, Update, Delete) standar (`index`, `create`, `store`, `show`, `edit`, `update`, `destroy`).

Perintah tunggal ini akan menghasilkan:

1. `app/Models/Product.php` (Model)
2. `app/Http/Controllers/ProductController.php` (Resource Controller)
3. `database/migrations/[timestamp]_create_products_table.php` (Migrasi)

Table perbandingan fungsi flag pada saat membuat model :

Flag Combination	Model	Migration	Controller Type	Factory & Seeder	Form Requests	Policy
-mcr	Yes	Yes	Web Resource (7 methods)	No	No	No
-a	Yes	Yes	Web Resource (7 methods)	Yes	Yes	Yes
-mc --api	Yes	Yes	API Resource (5 methods)	No	No	No

2) Struktur Model dalam Laravel

Berikut adalah contoh dasar Model Laravel:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    use HasFactory;

    protected $table = 'product';
    // Nama tabel dalam database (jika tidak sesuai dengan konvensi Laravel)

    protected $fillable = ['nama', 'harga', 'stok'];
    // Mass Assignment Protection
}
```

Penjelasan:

- `use HasFactory;` : Digunakan untuk mendukung fitur Factory dalam Laravel.
- `protected $table = 'product';` : Menentukan nama tabel di database (opsional, jika tabel tidak mengikuti konvensi Laravel).
- `protected $fillable = [...]` : Daftar kolom yang bisa diisi secara massal (mass assignment).

3) Konvensi Penamaan dalam Laravel Model

Laravel menggunakan konvensi default untuk model dan tabel:

- Nama model : dalam bentuk singular (tunggal), misalnya Product.
- Nama tabel : dalam bentuk plural (jamak), misalnya products.
- Nama kolom ID : default adalah id.

Jika ingin mengubah nama default kolom ID, bisa gunakan:

```
protected $primaryKey = 'id_product';
```

Jika primary key bukan auto-increment, tambahkan:

```
public $incrementing = false;
```

Dan jika bukan integer:

```
protected $keyType = 'string';
```

4) Eloquent ORM dan Query Builder dalam Model

a. Menyimpan Data ke Database

```
$product = new Product();  
$product->nama = "Laptop";  
$product->harga = 15000000;  
$product->stok = 10;  
$product->save();
```

Atau dengan metode mass assignment:

```
Product::create([  
    'nama' => 'Laptop',  
    'harga' => 15000000,  
    'stok' => 10  
]);
```

Pastikan kolom dalam database masuk dalam \$fillable.

b. Mengambil Data dari Database

```
$product = Product::all(); // Mengambil semua data  
$product = Product::find(1); // Mengambil data dengan ID 1  
$product = Product::where('harga', '>', 1000000)->get();  
// Mengambil product dengan harga > 1 juta
```

c. Mengupdate Data

```
$product = Product::find(1);  
$product->harga = 16000000;  
$product->save();
```

Atau dengan mass update:

```
Product::where('id', 1)->update(['harga' => 16000000]);
```

d. Menghapus Data

```
$product = Product::find(1);  
$product->delete();
```

Atau langsung:

```
Product::destroy(1);  
Product::where('harga', '<', 50000000)->delete();
```

5) Relasi dalam Laravel Model

Laravel Eloquent mendukung berbagai jenis hubungan antar tabel.

a. One to One

```
class User extends Model {  
    public function profile() {  
        return $this->hasOne(Profile::class);  
    }  
}
```

b. One to Many

```
class Kategori extends Model {  
    public function book() {  
        return $this->hasMany(Book::class);  
    }  
}
```

c. Many to Many

```
class Book extends Model {  
    public function kategori() {  
        return $this->belongsToMany(Kategori::class);  
    }  
}
```

d. Polymorphic Relationship

```
class Gambar extends Model {  
    public function imageable() {  
        return $this->morphTo();  
    }  
}
```

g. Hasil dan Pembahasan**Format Laporan**

https://docs.google.com/document/d/1FmnUT_6445Z7Ek8V1bqg0wfy8XVL_r3doVncwfuQfFU

- Kertas A4
- Format *.pdf
- Struktur Laporan: Disesuaikan dengan format yang diberikan dan ditambahkan foto resmi.
- Berikan Identitas Diri: NIM, Nama, Golongan sesuai yang tertera di Format
- Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
**Contoh: **ACARA-14_A_E1234356_BUDI.pdf
- Kumpulkan di Elearning : <https://slearn-jti.com/>

h. Kesimpulan

Dengan memahami konsep model dalam laravel, membuat dan mengelola model di laravel, melakukan operasi database menggunakan model, menerapkan relasi antar model dalam laravel, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan langkah kerja kurang tepat dalam mengerjakan		V		
4	Keterlambatan waktu pengumpulan dan langkah kerja tidak sesuai	V			
Total Skor*		Skor x 25			

ACARA 15. Migration

Pokok Bahasan	: Migration
Acara Praktikum	: MINGGU 4/3
Tempat	: Hybrid di Politeknik Negeri Jember
Alokasi Waktu	: 100 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep laravel migration.
2. Mahasiswa mampu menggunakan laravel migration untuk pengelolaan database.
3. Mahasiswa mampu menerapkan relasi antar tabel dengan laravel migration.
4. Mahasiswa mampu mengelola migrasi database secara efektif.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat LunakMenganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami konsep laravel migration.
2. Mahasiswa menggunakan laravel migration untuk pengelolaan database.
3. Mahasiswa menerapkan relasi antar tabel dengan laravel migration.
4. Mahasiswa mengelola migrasi database secara efektif.

d. Dasar Teori

Laravel Migration adalah fitur dalam framework Laravel yang memungkinkan pengelolaan skema database menggunakan skrip PHP daripada mengelola secara manual melalui SQL. Migration berfungsi sebagai sistem versi database, mirip dengan version control pada kode, sehingga memungkinkan pengembang untuk membuat, mengubah, dan berbagi struktur database dengan mudah. Mengapa harus menggunakan Migration?

1. Memudahkan Tim Pengembang

Dengan migration, pengembang bisa mengelola perubahan skema database secara terstruktur dan dapat dibagikan ke tim tanpa perlu berbagi file SQL secara manual.

2. Version Control untuk Database

Setiap perubahan dalam skema database terdokumentasi, sehingga memudahkan rollback ke versi sebelumnya.

3. Automasi Pembuatan Tabel dan Struktur Database

Laravel Migration memungkinkan pembuatan tabel, indeks, dan foreign key dengan cepat melalui skrip PHP.

4. Konsistensi antara Lingkungan Pengembangan

Dengan migration, database dapat disinkronkan di berbagai lingkungan (local, staging, production) tanpa kesalahan konfigurasi.

e. Alat dan Bahan

1. PC / Laptop
2. Windows 11
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Visual Studio Code
6. Kertas A4 / Folio Bergaris
7. Pulpen

f. Prosedur Kerja

1) Membuat Migration

Untuk membuat file migration, gunakan perintah artisan berikut:

```
php artisan make:migration create_users_table
```

Perintah ini akan membuat file baru di dalam direktori database/migrations/. Nama file akan mengikuti format timestamp agar Laravel tahu urutan eksekusi migration.

2) Struktur File Migration

Setiap file migration memiliki dua metode utama:

- `up()` : Digunakan untuk mendefinisikan perubahan skema yang akan diterapkan ke database.
- `down()` : Digunakan untuk membatalkan perubahan (rollback).

Contoh file migration (create_users_table.php):

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(){
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->string('password');
            $table->timestamps();
        });
    }

    public function down(){
        Schema::dropIfExists('users');
    }
};
```

Penjelasan:

- `Schema::create('users', function (Blueprint $table) { ... })` : Membuat tabel users.
- `$table->id();` : Membuat kolom id sebagai primary key dengan tipe data bigint auto-increment.
- `$table->string('name');` : Membuat kolom name dengan tipe data VARCHAR.
- `$table->string('email')->unique();` : Membuat kolom email dengan constraint UNIQUE.
- `$table->timestamps();` : Membuat kolom created_at dan updated_at otomatis.

3) Menjalankan Migration

Untuk mengeksekusi migration dan membuat tabel di database, gunakan perintah:

```
php artisan migrate
```

Perintah ini akan menjalankan semua migration yang belum dijalankan sebelumnya.

4) Membatalkan atau Mengulang Migration

Jika ingin membatalkan migration dan menghapus tabel yang telah dibuat, gunakan:

```
php artisan migrate:rollback
```

Untuk membatalkan dan menjalankan ulang semua migration:

```
php artisan migrate:refresh
```

Untuk mengulang semua migration dan mengisi ulang database dengan data seeder:

```
php artisan migrate:fresh --seed
```

5) Menambahkan Kolom ke Tabel yang Sudah Ada

Jika ingin menambahkan kolom baru ke tabel yang sudah ada, buat migration baru:

```
php artisan make:migration add_phone_to_users_table --table=users
```

Lalu edit file migration yang dibuat:

```
public function up()
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('phone')->nullable();
    });
}

public function down()
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn('phone');
    });
}
```

Jalankan migration:

```
php artisan migrate
```

6) Menggunakan Foreign Key dan Relasi

Jika ingin menambahkan foreign key, gunakan foreign() seperti berikut:

```
public function up()
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->text('content');
        $table->unsignedBigInteger('user_id');
        $table->foreign('user_id')->references('id')
            ->on('users')->onDelete('cascade');
        $table->timestamps();
    });
}
```

Foreign key ini akan menghubungkan user_id dengan kolom id di tabel users, dan jika pengguna dihapus, maka semua postingannya juga akan ikut terhapus (cascade).

g. Hasil dan Pembahasan**Format Laporan**

https://docs.google.com/document/d/1FmnUT_6445Z7Ek8V1bqg0wfy8XVL_r3doVncwfuQfFU

- Kertas A4
- Format *.pdf
- Struktur Laporan: Disesuaikan dengan format yang diberikan dan ditambahkan foto resmi.
- Berikan Identitas Diri: NIM, Nama, Golongan sesuai yang tertera di Format
- Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
**Contoh: **ACARA-15_A_E1234356_BUDI.pdf
- Kumpulkan di Elearning : <https://slearn-jti.com/>

h. Kesimpulan

Dengan memahami konsep laravel migration, menggunakan laravel migration untuk pengelolaan database, menerapkan relasi antar tabel dengan laravel migration, mengelola migrasi database secara efektif, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan langkah kerja kurang tepat dalam mengerjakan		V		
4	Keterlambatan waktu pengumpulan dan langkah kerja tidak sesuai	V			
Total Skor*		Skor x 25			

ACARA 16. Seeder dan Factories

Pokok Bahasan	: Seeder dan Factories
Acara Praktikum	: MINGGU 4/4
Tempat	: Hybrid di Politeknik Negeri Jember
Alokasi Waktu	: 100 Menit

a. Capaian Pembelajaran Mata Kuliah (CPMK)

1. Mahasiswa mampu memahami konsep dasar Seeder dan Factories dalam pengisian database.
2. Mahasiswa mampu menggunakan seeder untuk pengisian data awal.
3. Mahasiswa mampu menggunakan factories untuk membuat data dummy.
4. Mahasiswa mampu mengimplementasikan seeder dan factories dalam proyek Laravel.

b. Penilaian Bertahap BNSP (Skill Passport)

Nama Skema Sertifikasi : Pengembangan Web/Web Developer (SKM-496-028)

No.	Kode Unit	Nama Unit Kompetensi	Elemen Kompetensi
1	J.620100.001.01	Menganalisis Tools	• Mengidentifikasi tools yang akan digunakan • Menggunakan tools perangkat lunak
2	J.620100.002.01	Menganalisis Skalabilitas Perangkat LunakMenganalisis Skalabilitas Perangkat Lunak	• Menggunakan tools perangkat lunak • Menganalisis kebutuhan skalabilitas

c. Indikator Penilaian

1. Mahasiswa memahami konsep dasar Seeder dan Factories dalam pengisian database.
2. Mahasiswa menggunakan seeder untuk pengisian data awal.
3. Mahasiswa menggunakan factories untuk membuat data dummy.
4. Mahasiswa mengimplementasikan seeder dan factories dalam proyek Laravel.

d. Dasar Teori

Seeder digunakan untuk mengisi tabel dalam database dengan data awal atau dummy. Seeder biasanya digunakan dalam tahap pengembangan untuk menambahkan data default agar aplikasi bisa langsung diuji tanpa perlu memasukkan data secara manual.

Factories digunakan untuk membuat data dummy dalam jumlah besar dengan lebih fleksibel dan dinamis. Biasanya digunakan dalam kombinasi dengan **Seeder** untuk membuat banyak data dengan lebih cepat.

e. Alat dan Bahan

1. PC / Laptop
2. Windows 11
3. RAM 4 GB (Minimal) / 8 GB (Rekomendasi)
4. SSD 512GB
5. Visual Studio Code
6. Kertas A4 / Folio Bergaris
7. Pulpen

f. Prosedur Kerja

1) Cara Membuat dan Menggunakan Seeder

a) Membuat Seeder

Gunakan perintah Artisan berikut untuk membuat Seeder:

```
php artisan make:seeder NamaSeeder
```

Contoh:

```
php artisan make:seeder UsersTableSeeder
```

Perintah ini akan membuat file Seeder di:

```
database/seeder/UsersTableSeeder.php
```

b) Menulis Logika Seeder

Buka file UsersTableSeeder.php dan tambahkan kode berikut:

```
namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Str;
use Illuminate\Support\Facades\Hash;

class UsersTableSeeder extends Seeder
{
    public function run()
    {
        DB::table('users')->insert([
            'name' => 'Admin User',
            'email' => 'admin@example.com',
            'password' => Hash::make('password'),
        ]);
    }
}
```

c) Menjalankan Seeder

Untuk menjalankan Seeder, gunakan perintah:

```
php artisan db:seed --class=UsersTableSeeder
```

Jika ingin menjalankan semua Seeder sekaligus, jalankan:

```
php artisan db:seed
```

2) Cara Membuat dan Menggunakan Factory**a) Membuat Factory**

Gunakan perintah:

```
php artisan make:factory NamaFactory --model=NamaModel
```

Contoh:

```
php artisan make:factory UserFactory --model=User
```

File akan dibuat di:

database/factories/UserFactory.php

b) Menulis Logika Factory

Buka UserFactory.php dan isi dengan:

```
namespace Database\Factories;

use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Str;
use Illuminate\Support\Facades\Hash;

class UserFactory extends Factory
{
    protected $model = User::class;
    public function definition()
    {
        return [
            'name' => $this->faker->name,
            'email' => $this->faker->unique()->safeEmail,
            'password' => Hash::make('password'),
            'remember_token' => Str::random(10),
        ];
    }
}
```

c) Menggunakan Factory untuk Membuat Data

Untuk membuat satu user:

```
User::factory()->create();
```

Untuk membuat 10 user:

```
User::factory()->count(10)->create();
```

d) Menggunakan Factory dalam Seeder

Buka UsersTableSeeder.php dan ubah metode run():

```
use App\Models\User;

class UsersTableSeeder extends Seeder
{
    public function run()
    {
        User::factory()->count(50)->create();
        // Membuat 50 user
    }
}
```

e) Menjalankan Seeder dengan Factory

Jalankan perintah:

```
php artisan db:seed --class=UsersTableSeeder
```

f) Perbedaan Seeder dan Factory

Fitur	Seeder	Factory
Tujuan	Mengisi database dengan data awal atau data spesifik	Membuat data dummy dalam jumlah besar
Cara Kerja	Menggunakan DB::table()->insert() atau Model Eloquent	Menggunakan Faker untuk menghasilkan data secara otomatis
Cocok untuk	Data yang bersifat tetap seperti peran pengguna atau kategori produk	Data uji seperti pengguna, artikel, atau komentar dalam jumlah besar

g. Hasil dan Pembahasan

Format Laporan

https://docs.google.com/document/d/1FmnUT_6445Z7Ek8V1bqg0wfy8XVL_r3doVncwfuQfFU

- Kertas A4
- Format *.pdf
- Struktur Laporan: Disesuaikan dengan format yang diberikan dan ditambahkan foto resmi.
- Berikan Identitas Diri: NIM, Nama, Golongan sesuai yang tertera di Format
- Penamaan File: ACARA-X_GOL_NIM_NAMA.pdf
 **Contoh: **ACARA-16_A_E1234356_BUDI.pdf
- Kumpulkan di Elearning : <https://slearn-jti.com/>

h. Kesimpulan

Dengan memahami konsep dasar Seeder dan Factories dalam pengisian database, menggunakan seeder untuk pengisian data awal, menggunakan factories untuk membuat data dummy, mengimplementasikan seeder dan factories dalam proyek laravel, mahasiswa dapat melanjutkan proses praktikum ke tahap berikutnya.

i. Rubrik Penilaian

No	Indikator Penilaian	Nilai			
		1	2	3	4
1	Ketepatan waktu dan ketepatan langkah kerja dalam mengerjakan				V
2	Ketepatan waktu dan langkah kerja kurang tepat dalam mengerjakan			V	
3	Keterlambatan waktu dan langkah kerja kurang tepat dalam mengerjakan		V		
4	Keterlambatan waktu pengumpulan dan langkah kerja tidak sesuai	V			
Total Skor*		Skor x 25			

Tugas & Evaluasi Praktikum

Langkah 1: Persiapan Database

Pastikan file .env Anda sudah terkonfigurasi dengan benar seperti pada Pertemuan 1, dan database db_pos_toko sudah dibuat di MySQL Anda.

Langkah 2: Membuat Migration untuk Tabel Kategori dan Produk

Kita akan membangun relasi *One-to-Many* sederhana: Satu kategori (misal: "Makanan") bisa memiliki banyak produk (misal: "Beras", "Mie Instan").

1. Buka terminal di folder project pos-toko, lalu jalankan perintah Artisan berikut untuk membuat model beserta file migrasinya sekaligus:

```
php artisan make:model Category -m
```

```
php artisan make:model Product -m
```

Opsi -m otomatis membuat file migrasi baru di dalam folder database/migrations/.

2. Buka file migrasi kategori database/migrations/xxxx_xx_xx_XXXXXX_create_categories_table.php dan sesuaikan metode up-nya:

```
public function up(): void
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();

        $table->string('name')->unique();
        // Nama kategori (Makanan, Minuman, Kosmetik, dll)

        $table->string('slug')->unique();
        // URL ramah SEO (makanan, minuman)

        $table->timestamps();
    });
}
```

3. Buka file migrasi produk

database/migrations/xxxx_xx_xx_XXXXXX_create_products_table.php dan definisikan kolom serta *foreign key* relasinya:

```
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        // Menghubungkan ke tabel categories

        $table->foreignId('category_id')
            ->constrained()->onDelete('cascade');

        $table->string('name');

        $table->string('sku')->unique();
        // Stock Keeping Unit / Kode Barcode Produk

        $table->integer('price');
        // Harga jual barang

        $table->integer('stock')->default(0);

        $table->timestamps();
    });
}
```

4. Jalankan migrasi tersebut agar tabel terbentuk di database MySQL Anda:

```
php artisan migrate
```

Periksa database Anda via phpMyAdmin untuk memastikan tabel categories dan products telah sukses terbuat.

Langkah 3 : Mengonfigurasi Model (Mass Assignment)

Sebelum mengisi data, kita perlu memberi tahu Laravel kolom mana saja yang boleh diisi secara massal (*Mass Assignment*).

1. Buka app/Models/Category.php dan tambahkan properti \$fillable:

```
protected $fillable = ['name', 'slug'];
```

2. Buka app/Models/Product.php dan tambahkan properti \$fillable:

```
protected $fillable = ['category_id', 'name', 'sku', 'price', 'stock'];
```

Langkah 4 : Membuat dan Menjalankan Seeder Kategori

Kita akan membuat data kategori tetap untuk toko kelontong kita.

1. Jalankan perintah Artisan berikut:

```
php artisan make:seeder CategorySeeder
```

2. Buka file database/seeder/CategorySeeder.php lalu ubah method run() menjadi:

```
namespace Database\Seeders;

use App\Models\Category;
use Illuminate\Database\Seeder;
use Illuminate\Support\Str;

class CategorySeeder extends Seeder
{
    public function run(): void
    {
        $categories = [
            'Makanan',
            'Minuman',
            'Kebutuhan Rumah Tangga',
            'Obat-obatan',
        ];

        foreach ($categories as $cat) {
            Category::create([
                'name' => $cat,
                'slug' => Str::slug($cat)
            ]);
        }
    }
}
```

Langkah 5: Membuat dan Menjalankan Factory Produk

Untuk mengisi tabel produk dengan puluhan data tiruan, kita memanfaatkan Factory.

1. Jalankan perintah pembuatan factory:

```
php artisan make:factory ProductFactory
```

2. Buka file database/factories/ProductFactory.php dan gunakan library Faker untuk menyusun format data dummy:

```
namespace Database\Factories;

use App\Models\Category;
use Illuminate\Database\Eloquent\Factories\Factory;

class ProductFactory extends Factory
{
    public function definition(): void
    {
        return [
            // Mengambil ID kategori secara acak dari data yang sudah ada
            'category_id' => Category::inRandomOrder()->first()->id ?? 1,
            'name' => $this->faker->words(2, true),
            // Menghasilkan 2 kata nama produk
            'sku' => 'PRD-' . $this->faker->unique()
                ->numberBetween(10000, 99999),
            'price' => $this->faker->numberBetween(2000, 50000),
            // Rentang harga barang
            'stock' => $this->faker->numberBetween(5, 100),
            // Rentang jumlah stok
        ];
    }
}
```

Langkah 6: Menggabungkan Eksekusi di DatabaseSeeder

Buka file master seeder di database/seeder/DatabaseSeeder.php. File ini yang mengontrol urutan pengisian database. Ubah isinya menjadi:

```
public function run(): void
{
    // 1. Jalankan Seeder Kategori terlebih dahulu
    // agar data kategorinya ada
    $this->call(CategorySeeder::class);

    // 2. Jalankan Factory Produk untuk membuat 50
    // data dummy produk
    \App\Models\Product::factory(50)->create();
}
```

Jalankan perintah pengisian database secara menyeluruh:

```
php artisan db:seed
```

Buka tabel products di database Anda. Anda akan melihat 50 baris data barang kelontong acak siap pakai lengkap dengan relasi kategori, SKU unik, harga, dan stok.

Selesaikan tugas mandiri berikut untuk memantapkan pemahaman Anda:

1. Buat satu migrasi tabel baru bernama suppliers dengan kolom: id, name (nama perusahaan supplier), phone (nomor telepon), dan address (alamat).

2. Terapkan perintah `php artisan migrate` untuk memasukkan tabel tersebut ke database.
3. Buat file `SupplierSeeder` untuk memasukkan minimal 3 data supplier riil secara manual (misal: PT. Indofood, PT. Unilever).
4. Daftarkan `SupplierSeeder` ke dalam `DatabaseSeeder.php` lalu jalankan perintah `php artisan migrate:fresh --seed` (Hati-hati: perintah ini akan menghapus ulang seluruh database dan mengisinya kembali dari awal).
5. Kumpulkan screenshot struktur tabel `suppliers` di `phpMyAdmin` beserta isi datanya dalam bentuk file PDF.